Repeated Square-and-Multiply Method

Efficient Modular Exponentiation

Problem Statement

We want to efficiently compute:

$$y = x^e \bmod n$$

- Direct computation of x^e is infeasible for large e
- Commonly used in cryptography (e.g. RSA, Diffie–Hellman)
- The square-and-multiply method reduces complexity to $O(\log e)$

Idea of Square-and-Multiply

- Write the exponent e in binary
- Process bits from left to right
- Repeatedly:
  - Square the current value
  - Multiply by x if the current bit is 1
  - Reduce modulo n at every step

Algorithm (Left-to-Right)

Input: Integers x, $e = (e_k \dots e_1 e_0)_2$, modulus n

Output: $y = x^e \bmod n$

1. Set $y \leftarrow 1$
2. For $i = k$ down to 0:
   1. $y \leftarrow y^2 \bmod n$
   2. If $e_i = 1$, then $y \leftarrow y \cdot x \bmod n$
3. Return y

Worked Example

Compute:

$$y = 7^{13} \bmod 20$$

- $x = 7,\ e = 13,\ n = 20$
- Binary representation:

$$13 = (1101)_2$$

Example: Step-by-Step Computation

Initialize:

$$y = 1$$

Process bits of $e = 1101_2$ (left to right):

Bit	Operation	y mod 20
1	$y = 1^2 \cdot 7$	7
1	$y = 7^2 \cdot 7$	3
0	$y = 3^2$	9
1	$y = 9^2 \cdot 7$	7

Final Result

$$7^{13} \bmod 20 = \boxed{7}$$

- Only $O(\log e)$ squarings and multiplications
- Intermediate values always kept small using modulo n

Why This Matters

- Enables practical public-key cryptography
- Used in RSA encryption and signatures
- Efficient even for exponents with hundreds or thousands of bits